

Algorithm Analysis Fundamentals

Time complexity analysis of iterative and recursive algorithms; verify Big-O, Ω , Θ for sample programs

Dr Sriramy

Jaasiel J.S

Experiment 1: Linear Search (Iterative)

- **Objective:** To analyze time complexity of linear search.
- **Problem Statement:** Write a Python program to search for a key in an array using linear search.
- **Sample Input:** [10, 25, 30, 45, 50], Key = 30
- **Sample Output:** Key found at index 2
- **Complexity:** Ω : $O(1)$, O : $O(n)$, Θ : $\Theta(n)$
- **Explanation:** Demonstrates sequential scanning and inefficiency for large inputs.

main.py	Output
<pre>1 A=list(map(int,input().split())) 2 key=int(input("Enter the key element:\n")) 3 for i in range(len(A)): 4 if A[i]==key: 5 print("key found at index",i) 6 break 7 else: 8 print("not found")</pre>	<pre>10 25 30 45 50 Enter the key element: 30 key found at index 2 === Code Execution Successful ===</pre>

Experiment 2: Binary Search (Recursive)

- **Objective:** To compare recursive binary search complexity.
- **Problem Statement:** Implement recursive binary search in Python.
- **Sample Input:** [5, 10, 15, 20, 25], Key = 20
- **Sample Output:** Key found at index 3
- **Complexity:** Ω : $O(1)$, O : $O(\log_2 n)$, Θ : $\Theta(\log_2 n)$

- **Explanation:** Shows divide-and-conquer and logarithmic efficiency.

main.py	Output
<pre> 1 def search(a,low,high,key): 2 if low>high: 3 return -1 4 mid=(low+high)//2 5 if a[mid]==key: 6 return mid 7 elif key<a[mid]: 8 return search(a,low,mid-1,key) 9 else: 10 return search(a,mid+1,high,key) 11 12 a=list(map(int,input().split())) 13 key=int(input("Enter the key element:\n")) 14 result=search(a,0,len(a)-1,key) 15 if result ==-1: 16 print("key not found") 17 else: 18 print("key found at index",result) 19 </pre>	<pre> 5 10 15 20 25 Enter the key element: 20 key found at index 3 === Code Execution Successful === </pre>

Experiment 3: Factorial (Iterative vs Recursive)

- **Objective:** To compare iterative and recursive factorial computation.
- **Problem Statement:** Write programs to compute factorial using both methods.
- **Sample Input:** n = 5
- **Sample Output:** 120
- **Complexity:** Both $O(n)$
- **Explanation:** Highlights recursion depth vs iterative loops.

ITERATIVE METHOD

main.py	Output
<pre> 1 n = int(input("Enter a number: ")) 2 3 fact = 1 4 5 for i in range(1, n + 1): 6 fact = fact * i 7 8 print("Factorial =", fact) </pre>	<pre> Enter a number: 5 Factorial = 120 === Code Execution Successful === </pre>

RECURSIVE METHOD

main.py	Output
<pre>1 def factorial(n): 2 if n == 0 or n == 1: 3 return 1 4 return n * factorial(n - 1) 5 6 n = int(input("Enter a number: ")) 7 print("Factorial =", factorial(n))</pre>	<pre>Enter a number: 5 Factorial = 120 === Code Execution Successful ===</pre>

Experiment 4: Fibonacci Series

- **Objective:** To analyze recursive vs iterative Fibonacci generation.
- **Problem Statement:** Generate first n Fibonacci numbers.
- **Sample Input:** n = 6
- **Sample Output:** [0, 1, 1, 2, 3, 5]
- **Complexity:** Iterative $O(n)$, Recursive naïve $O(2n)$, Recursive with memoization $O(n)$
- **Explanation:** Illustrates exponential recursion vs efficient iteration.

ITERATIVE METHOD

main.py	Output
<pre>1 n = int(input("Enter n: ")) 2 3 a, b = 0, 1 4 5 for i in range(n): 6 print(a, end=" ") 7 a, b = b, a + b</pre>	<pre>Enter n: 6 0 1 1 2 3 5 === Code Execution Successful ===</pre>

```
main.py  [Full Screen] [Dark Theme] [Share] [Run] Output
1 def fibonacci(n):
2     if n <= 1:
3         return n
4     return fibonacci(n - 1) + fibonacci(n - 2)
5
6 n = int(input("Enter n: "))
7
8 for i in range(n):
9     print(fibonacci(i), end=" ")

Enter n: 6
0 1 1 2 3 5
=== Code Execution Successful ===
```

Experiment 5: Matrix Multiplication

- **Objective:** To analyze nested loop complexity.
- **Problem Statement:** Multiply two matrices using loops.
- **Sample Input:** $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$
- **Sample Output:** $\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$
- **Complexity:** $O(n^3)$
- **Explanation:** Shows cubic growth due to nested loops.

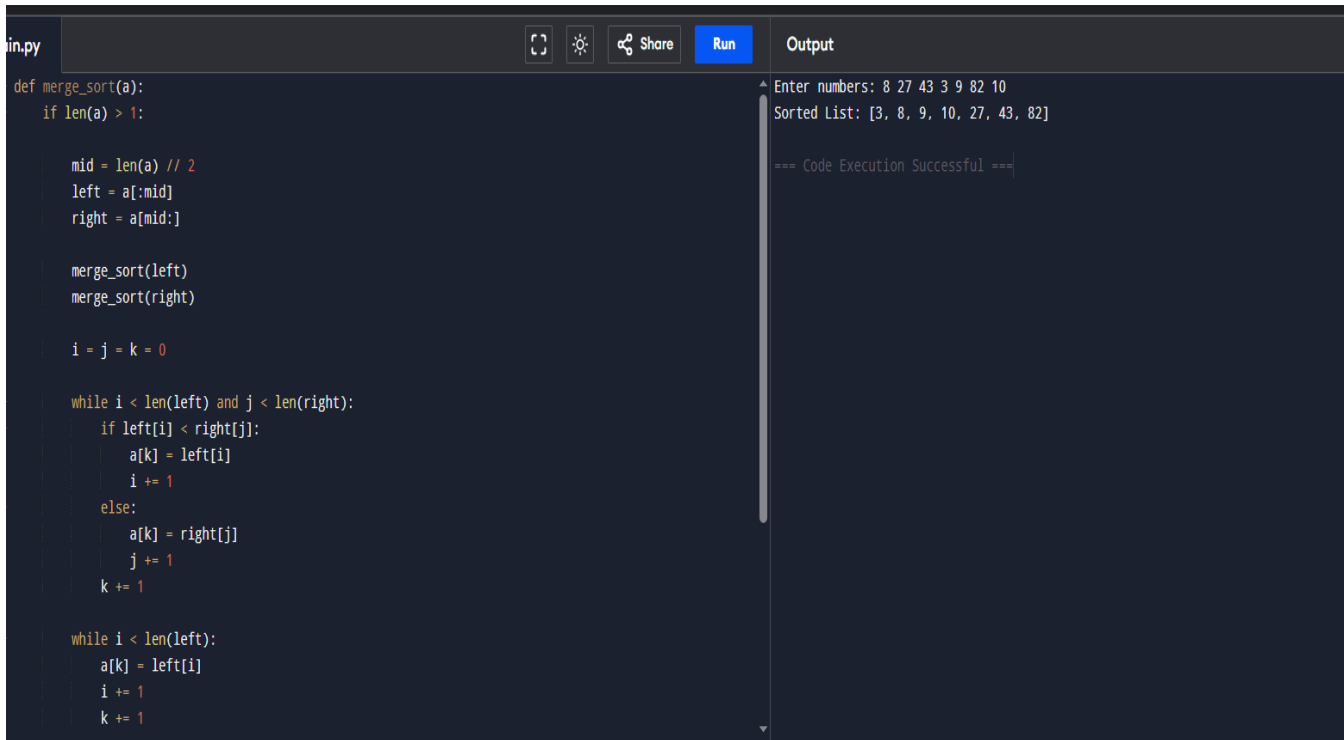
```
main.py  [Full Screen] [Dark Theme] [Share] [Run] Output
12 B = []
13 print("Enter elements of Matrix B:")
14 for i in range(r2):
15     B.append(list(map(int, input().split())))
16
17 C = []
18
19 for i in range(r1):
20     row = []
21     for j in range(c2):
22         s = 0
23         for k in range(c1):
24             s += A[i][k] * B[k][j]
25         row.append(s)
26     C.append(row)
27
28 print("Result Matrix:")
29 for row in C:
30     print(row)

Enter rows of Matrix A: 2
Enter columns of Matrix A: 2
Enter elements of Matrix A:
1 2
3 4
Enter rows of Matrix B: 2
Enter columns of Matrix B: 2
Enter elements of Matrix B:
5 6
7 8
Result Matrix:
[19, 22]
[43, 50]
=== Code Execution Successful ===
```

Experiment 6: Merge Sort

- **Objective:** To analyze divide-and-conquer sorting.
- **Problem Statement:** Implement merge sort.
- **Sample Input:** [38, 27, 43, 3, 9, 82, 10]

- **Sample Output:** [3, 9, 10, 27, 38, 43, 82]
- **Complexity:** Always $O(n \log n)$
- **Explanation:** Demonstrates guaranteed efficiency of divide-and-conquer.



```

def merge_sort(a):
    if len(a) > 1:
        mid = len(a) // 2
        left = a[:mid]
        right = a[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                a[k] = left[i]
                i += 1
            else:
                a[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            a[k] = left[i]
            i += 1
            k += 1

```

Output

Enter numbers: 8 27 43 3 9 82 10
 Sorted List: [3, 8, 9, 10, 27, 43, 82]
 === Code Execution Successful ===

Experiment 7: Quick Sort

- **Objective:** To analyze recursive quick sort.
- **Problem Statement:** Implement quick sort.
- **Sample Input:** [10, 7, 8, 9, 1, 5]
- **Sample Output:** [1, 5, 7, 8, 9, 10]
- **Complexity:** Ω : $O(n \log n)$, O : $O(n^2)$, Θ : $\Theta(n \log n)$
- **Explanation:** Shows how pivot choice affects performance.

main.py	Run	Output
<pre>1 def quick_sort(a): 2 if len(a) <= 1: 3 return a 4 5 pivot = a[0] 6 7 left = [x for x in a[1:] if x <= pivot] 8 right = [x for x in a[1:] if x > pivot] 9 10 return quick_sort(left) + [pivot] + quick_sort(right) 11 12 a = list(map(int, input("Enter numbers: ").split())) 13 14 print("Sorted List:", quick_sort(a))</pre>		<pre>Enter numbers: 0 7 8 9 1 5 Sorted List: [0, 1, 5, 7, 8, 9] === Code Execution Successful ===</pre>

Experiment 8: Tower of Hanoi

- **Objective:** To analyze exponential recursion.
- **Problem Statement:** Solve Tower of Hanoi for n disks.
- **Sample Input:** n = 3
- **Sample Output:** Sequence of moves.
- **Complexity:** $O(2^n)$
- **Explanation:** Illustrates exponential growth in recursive calls.

main.py	Run	Output
<pre>1 def hanoi(n, source, auxiliary, destination): 2 if n == 1: 3 print("Move disk 1 from", source, "to", destination) 4 return 5 6 hanoi(n - 1, source, destination, auxiliary) 7 print("Move disk", n, "from", source, "to", destination) 8 hanoi(n - 1, auxiliary, source, destination) 9 10 n = int(input("Enter number of disks: ")) 11 12 hanoi(n, 'A', 'B', 'C')</pre>		<pre>Enter number of disks: 3 Move disk 1 from A to C Move disk 2 from A to B Move disk 1 from C to B Move disk 3 from A to C Move disk 1 from B to A Move disk 2 from B to C Move disk 1 from A to C === Code Execution Successful ===</pre>

Experiment 9: GCD (Euclidean Algorithm)

- **Objective:** To analyze recursive GCD.
- **Problem Statement:** Implement Euclidean algorithm.
- **Sample Input:** a=48, b=18
- **Sample Output:** 6
- **Complexity:** $O(\log \min(a, b))$
- **Explanation:** Shows efficient recursion with logarithmic complexity.

main.py	Run	Output
<pre>1 def gcd(a, b): 2 if b == 0: 3 return a 4 return gcd(b, a % b) 5 6 a = int(input("Enter first number: ")) 7 b = int(input("Enter second number: ")) 8 9 print("GCD =", gcd(a, b))</pre>		<pre>Enter first number: 48 Enter second number: 18 GCD = 6 === Code Execution Successful ===</pre>

Experiment 10: Insertion Sort

- **Objective:** To analyze iterative sorting.
- **Problem Statement:** Implement insertion sort.
- **Sample Input:** [12, 11, 13, 5, 6]
- **Sample Output:** [5, 6, 11, 12, 13]
- **Complexity:** $\Omega: O(n)$, $O: O(n^2)$, $\Theta: \Theta(n^2)$
- **Explanation:** Demonstrates input sensitivity (sorted vs unsorted).

```
main.py  [ ] [ ] [ ] Share Run Output
2
3 for i in range(1, len(a)):
4     key = a[i]
5     j = i - 1
6
7     while j >= 0 and a[j] > key:
8         a[j + 1] = a[j]
9         j -= 1
10
11     a[j + 1] = key
12
13 print("Sorted List:", a)
```

Enter numbers: 2 11 13 5 6
Sorted List: [2, 5, 6, 11, 13]
=== Code Execution Successful ===

Experiment 11: Heap Sort

- **Objective:** To analyze heap-based sorting.
- **Problem Statement:** Implement heap sort.
- **Sample Input:** [4, 10, 3, 5, 1]
- **Sample Output:** [1, 3, 4, 5, 10]
- **Complexity:** Always $O(n \log n)$
- **Explanation:** Consistent performance using heap structure.

```
main.py  [ ] [ ] [ ] Share Run Output
15
16 a = list(map(int, input("Enter numbers: ").split()))
17 n = len(a)
18
19 for i in range(n // 2 - 1, -1, -1):
20     heapify(a, n, i)
21
22 for i in range(n - 1, 0, -1):
23     a[0], a[i] = a[i], a[0]
24     heapify(a, i, 0)
25
26 print("Sorted List:", a)
```

Enter numbers: 4 10 3 5 1
Sorted List: [1, 3, 4, 5, 10]
=== Code Execution Successful ===

Experiment 12: Counting Inversions

- **Objective:** To analyze modified merge sort.
- **Problem Statement:** Count inversions in an array.
- **Sample Input:** [2, 4, 1, 3, 5]
- **Sample Output:** 3
- **Complexity:** $O(n \log n)$
- **Explanation:** Shows divide-and-conquer applied beyond sorting.

```
main.py
1- def merge(a):
2-     if len(a) <= 1:
3-         return a, 0
4-
5-     mid = len(a) // 2
6-
7-     left, inv1 = merge(a[:mid])
8-     right, inv2 = merge(a[mid:])
9-
10-    result = []
11-    i = j = inv = 0
12-
13-    while i < len(left) and j < len(right):
14-        if left[i] <= right[j]:
15-            result.append(left[i])
16-            i += 1
17-        else:
18-            result.append(right[j])
19-            inv += len(left) - i
20-            j += 1
21-
22-    result.extend(left[i:])
23-    result.extend(right[j:])
24-
25-    return result, inv + inv1 + inv2
26-
27-
28- a = list(map(int, input("Enter numbers: ").split()))
29-
30- sorted_array, count = merge(a)
31-
32- print("Number of Inversions =", count)
```

Output

Enter numbers: 2 4 1 3 5
Number of Inversions = 3

=== Code Execution Successful ===

Experiment 13: Maximum Subarray Sum

- **Objective:** To compare Kadane's vs divide-and-conquer.
- **Problem Statement:** Find maximum subarray sum.
- **Sample Input:** [-2, -3, 4, -1, -2, 1, 5, -3]
- **Sample Output:** 7
- **Complexity:** Kadane's $O(n)$, Divide & Conquer $O(n \log n)$
- **Explanation:** Highlights efficiency differences between approaches.

main.py	Run	Output
<pre>1 a = list(map(int, input("Enter numbers: ").split())) 2 3 current = maximum = a[0] 4 5 for i in range(1, len(a)): 6 current = max(a[i], current + a[i]) 7 maximum = max(maximum, current) 8 9 print("Maximum Subarray Sum =", maximum)</pre>		<pre>Enter numbers: 2 -3 4 -1 -2 1 5 -3 Maximum Subarray Sum = 7 === Code Execution Successful ===</pre>

Experiment 14: Exponentiation

- **Objective:** To compare iterative vs recursive fast power.
- **Problem Statement:** Compute x^n .
- **Sample Input:** $x=2, n=10$
- **Sample Output:** 1024
- **Complexity:** Iterative $O(n)$, Recursive fast power $O(\log n)$
- **Explanation:** Shows how recursion reduces complexity.

ITERATIVE METHOD

main.py	Run	Output
<pre>1 x = int(input("Enter base: ")) 2 n = int(input("Enter exponent: ")) 3 4 result = 1 5 6 for i in range(n): 7 result *= x 8 9 print("Answer =", result)</pre>		<pre>Enter base: 2 Enter exponent: 10 Answer = 1024 === Code Execution Successful ===</pre>

RECURSIVE METHOD

```
main.py  [Icons] [Share] [Run] Output
1 def power(x, n):
2     if n == 0:
3         return 1
4
5     if n % 2 == 0:
6         half = power(x, n // 2)
7         return half * half
8
9     else:
10        return x * power(x, n - 1)
11
12
13 x = int(input("Enter base: "))
14 n = int(input("Enter exponent: "))
15
16 print("Answer =", power(x, n))
```

Enter base: 2
Enter exponent: 10
Answer = 1024

=== Code Execution Successful ===

Experiment 15: Closest Pair of Points

- **Objective:** To analyze brute force vs divide-and-conquer.
- **Problem Statement:** Find closest pair of points in 2D.
- **Sample Input:** $[(1, 2), (4, 5), (7, 8), (3, 1)]$
- **Sample Output:** Closest pair $(1, 2) - (3, 1)$, Distance = $\sqrt{5}$
-

```
main.py  [Icons] [Share] [Run] Output
2
3 points = [(1, 2), (4, 5), (7, 8), (3, 1)]
4
5 min_dist = float('inf')
6 pair = ()
7
8 for i in range(len(points)):
9     for j in range(i + 1, len(points)):
10        x1, y1 = points[i]
11        x2, y2 = points[j]
12
13        dist = math.sqrt((x2 - x1)**2 + (y2 - y1)**2)
14
15        if dist < min_dist:
16            min_dist = dist
17            pair = (points[i], points[j])
18
19 print("Closest Pair:", pair)
20 print("Distance =", min_dist)
```

Closest Pair: ((1, 2), (3, 1))
Distance = 2.23606797749979

=== Code Execution Successful ===